

Portia — Trabajo Adicional Post-Revisión

Todo lo que se trabajó en paralelo a la revisión de Profe Salvador — mantenimiento, infraestructura y funcionalidad que no estaban entre sus 16 puntos, pero que salieron al auditar el proyecto de punta a punta.

Preparado: 03 de julio de 2026

Resumen ejecutivo

La revisión de Profe Salvador fue el punto de partida, pero al meterse a fondo en el repositorio para responderla aparecieron varios frentes que no estaban entre sus 16 puntos: dependencias completamente atrasadas, el sitio en producción sirviendo una versión vieja sin que nadie lo notara, una funcionalidad de tiempo real a medio construir, y accesos de cuenta que valía la pena ordenar antes de seguir.

Este documento junta ese trabajo aparte — el que no responde ningún punto puntual de la revisión, pero que dejó el proyecto más sólido igual.

Dependencias del proyecto

1. Actualización completa de dependencias

IMPLEMENTADO

SITUACIÓN

Había 20 Pull Requests de Dependabot abiertos y sin revisar, varios de versiones mayores (Next.js, React, Electron, Tailwind, Vitest, electron-builder) que llevaban semanas acumulándose sin que nadie se animara a tocarlos.

QUÉ SE HIZO

Se revisaron los 20, uno por uno. Los de bajo riesgo (parches, GitHub Actions) se mergearon directo. Los seis cambios de versión mayor se probaron de verdad antes de aceptarlos: se levantó la app real, se probó con login de un conserje y de un administrador, y en el caso de Electron se interactuó con la ventana nativa para confirmar que nada se rompió. Tailwind se sacó por completo en vez de actualizarlo — no se usaba ni una sola clase suya en todo el código, así que migrar su configuración solo para mantenerlo al día no tenía sentido.

Infraestructura y despliegue

2. El panel de administración llevaba semanas desactualizado en producción

IMPLEMENTADO

SITUACIÓN

El sitio en vivo (admin-five-bay-95.vercel.app) no reflejaba los últimos cambios del código. Nadie lo había notado porque nadie comparó el sitio real contra el repositorio.

QUÉ SE HIZO

Se descubrió que Vercel nunca tuvo el auto-deploy conectado a GitHub: cada publicación anterior se había hecho a mano desde una terminal. Se conectó la integración real, y de paso se encontró un problema nuevo (Turbo no resolvía el workspace sin un campo de configuración que faltaba en el archivo raíz del proyecto) que rompía todos los despliegues automáticos apenas se activó la conexión. Ya está resuelto: cada push a la rama principal ahora publica solo, y se verificó con un despliegue real llegando a producción.

3. Acceso directo a Supabase y a Vercel

IMPLEMENTADO

SITUACIÓN

Cada operación sobre la base de datos o sobre el despliegue dependía de manejar la clave de servicio a mano o de usar la terminal de Vercel — lento y con más riesgo de exponer algo por accidente.

QUÉ SE HIZO

Se conectó acceso directo a ambas plataformas. Esto es lo que permitió, en la misma sesión, revisar el estado real de seguridad de la base de datos, aplicar correcciones directo en producción con trazabilidad, y verificar despliegues sin depender de comandos manuales.

Funcionalidad

4. El panel de administración no veía cambios en vivo

IMPLEMENTADO

SITUACIÓN

Solo la pantalla principal del panel (el resumen) se actualizaba sola cuando un conserje registraba algo. Las cuatro pantallas de detalle — novedades, visitas, encomiendas, tareas — solo se refrescaban si alguien cambiaba de filtro o de pestaña a mano.

QUÉ SE HIZO

Se armó un mecanismo compartido y se aplicó a las cuatro pantallas. Se probó en vivo: se creó una tarea de prueba desde el panel y apareció sola, sin recargar la página.

5. Retención de datos personales según la Ley 21.719

IMPLEMENTADO

SITUACIÓN

Las visitas ya se borraban automáticamente después de 30 días, pero las encomiendas y las novedades — que también tienen datos de personas — se quedaban para siempre en la base de datos.

QUÉ SE HIZO

Con la aprobación explícita de Diego sobre el plazo (90 días, igual que ya regía para otra tabla), se armaron y activaron dos tareas automáticas nuevas que corren todas las noches y borran los registros viejos. Antes de activarlas, se les sacó el permiso de ejecución pública — solo pueden dispararse desde el proceso interno, no desde afuera.

Seguridad de la base de datos

6. Dos funciones internas quedaban ejecutables desde afuera sin login

IMPLEMENTADO

SITUACIÓN

Al revisar el estado de seguridad real de la base de datos (algo que la revisión de Profe Salvador pedía en general, pero esto no era uno de sus puntos específicos), aparecieron dos funciones que cualquiera podía invocar directo por la API pública, sin necesitar sesión.

QUÉ SE HIZO

Se les sacó el permiso de ejecución a los roles públicos, dejándolas solo para el proceso interno del sistema. También se corrigió que tres funciones no tenían fijada su ruta de búsqueda interna — una falla de configuración conocida en Postgres que en teoría podría explotarse para hacer que una función ejecute código de otro lugar.

Orden y acceso del equipo

7. No estaba claro si James tenía acceso real al repositorio

EN CURSO

SITUACIÓN

James había hecho commits en el pasado, lo que lo hace aparecer como “contribuidor” en GitHub — pero eso no es lo mismo que tener acceso de colaborador hoy. Se confirmó que no está agregado como colaborador: su último trabajo pasó por Pull Request desde su propia copia del repositorio, hace más de una semana.

QUÉ SE HIZO

Se dejó documentado el camino para agregarlo como colaborador con permiso de escritura, cosa de que en vez de trabajar en una copia aparte y quedar atrasado, trabaje directo sobre el mismo repositorio que todos.

***Nota:** Esta decisión queda pendiente de Diego — agregar a alguien como colaborador es una decisión de acceso que no se toma por otra persona.*

Conclusión

Nada de esto estaba en la lista de Profe Salvador, pero salió a la luz por la misma razón que sus 16 puntos: cuando alguien se mete de verdad a auditar un proyecto en vez de asumir que está bien, aparecen cosas. La actualización de dependencias y la conexión de despliegue eran deuda acumulada de semanas; el hallazgo de seguridad en la base de datos y el estado real de acceso de James eran cosas que nadie había mirado todavía con esta profundidad.

Lo único que sigue realmente abierto de este grupo es el acceso de James — y esa decisión es tuya, no técnica.